

ml

Tanvir

(data + learning algorithm) $\rightarrow$ function

1 Supervised learning

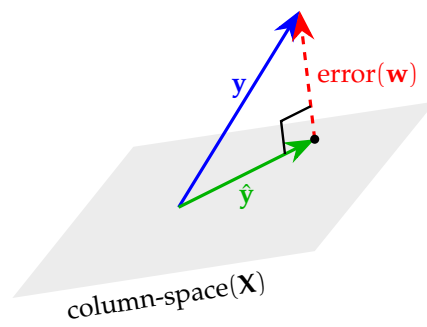
1.1 Regression

model, parameters, cost function, objective

1.1.1 Linear (in $\mathbf{w}$) Regression

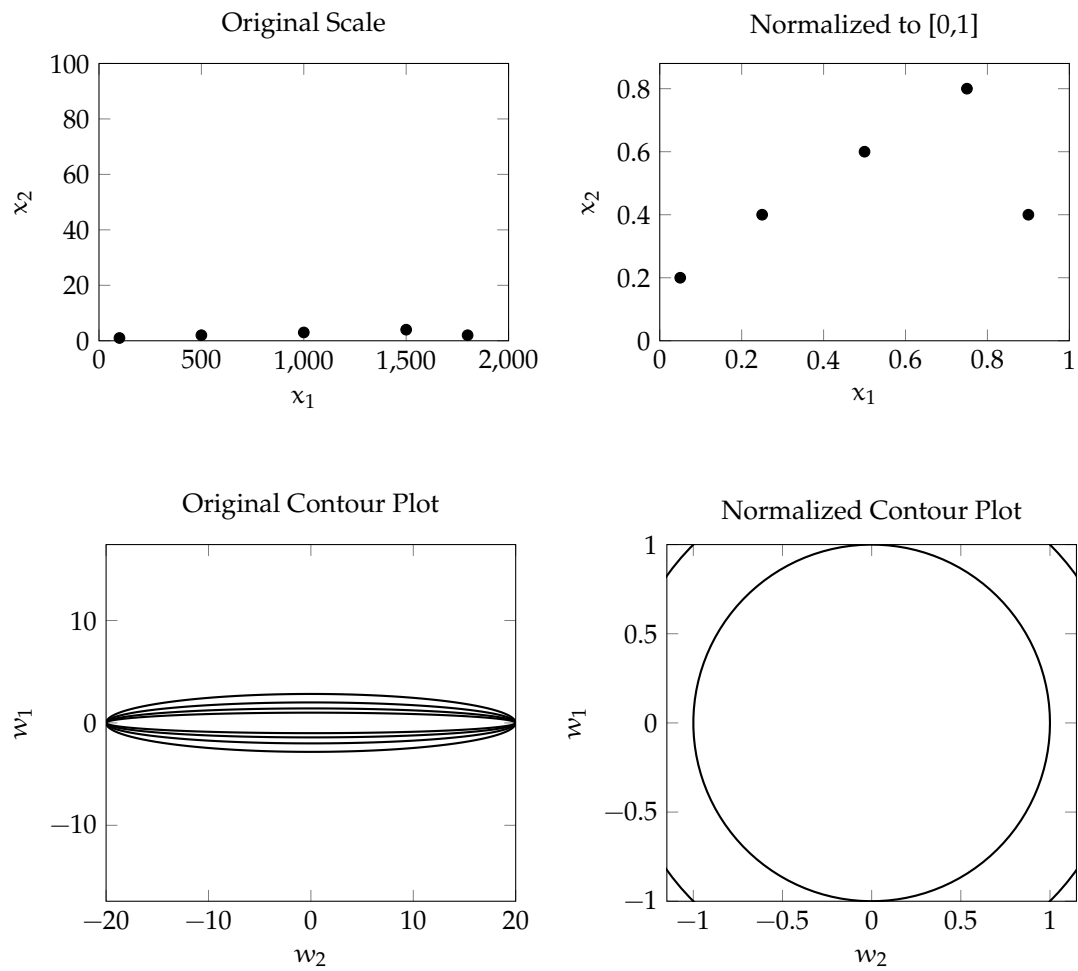
$\mathbf{X}^{m \times n}$ has m examples, each having n features. Usually $m \gg n$. $\mathbf{y}^{m \times 1}$ are the corresponding outputs.

$$\begin{aligned}\mathbf{X}\mathbf{w} &= \hat{\mathbf{y}} \\ \text{error}(\mathbf{w}) &= \mathbf{y} - \hat{\mathbf{y}} \\ \underset{\mathbf{w}}{\text{minimize}} \quad J(\mathbf{w}) &= \frac{1}{m} \|\text{error}(\mathbf{w})\|^2 = \frac{1}{m} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} &= \frac{2}{m} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ \text{gradient descent: } \mathbf{w} &= \mathbf{w} - \alpha \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}\end{aligned}$$



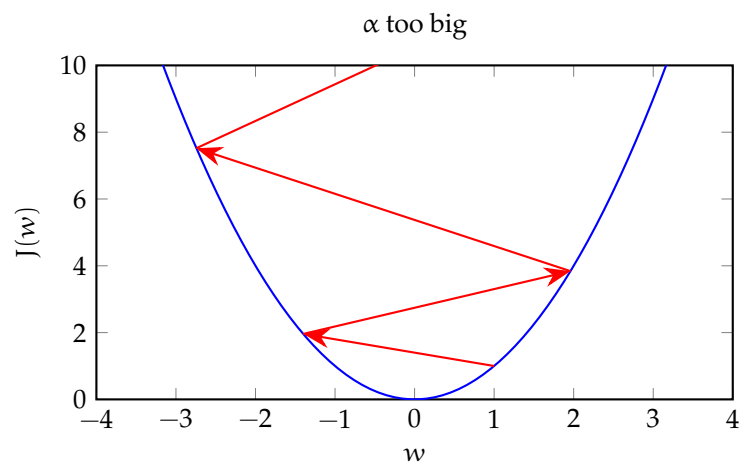
1.2 Normalization

Changes scale keeping the shape of distribution same. Gradient descent now works, with more ease, in the world of concentric circular contours.



1.3 Learning rate, α

A good value is somewhere between too big (divergence) and too small (slow convergence).



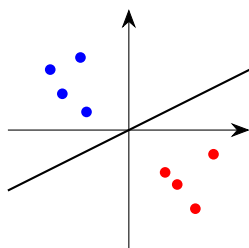
1.4 Classification

1.4.1 Logistic Regression

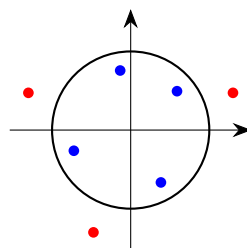
$$z = \mathbf{X}\mathbf{w}, \quad // \text{ linear regression}$$
$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad // \text{ maps } z \text{ to probability-like } (0,1)$$

At $z = 0$, $\sigma(z) = 0.5$. So, with threshold 0.5, $z = 0$ is the decision boundary. As linear regression doesn't have to be straight line, logistic regression's decision boundary does not have to be a straight line.

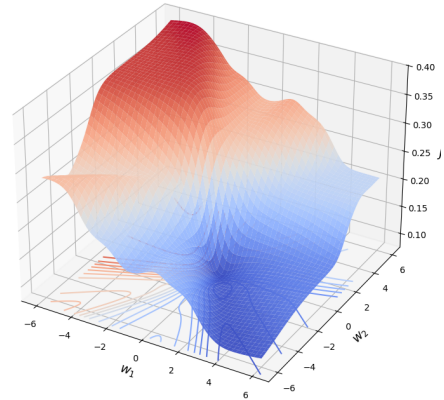
Linear boundary



Circular boundary



The squared error loss function $L(\mathbf{w}, y_i) = (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i)^2$, makes the cost function non-convex.

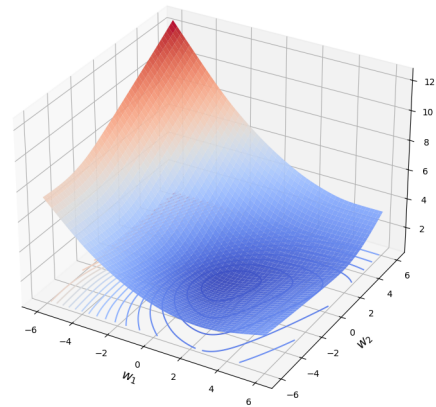


Negative log likelihood loss:

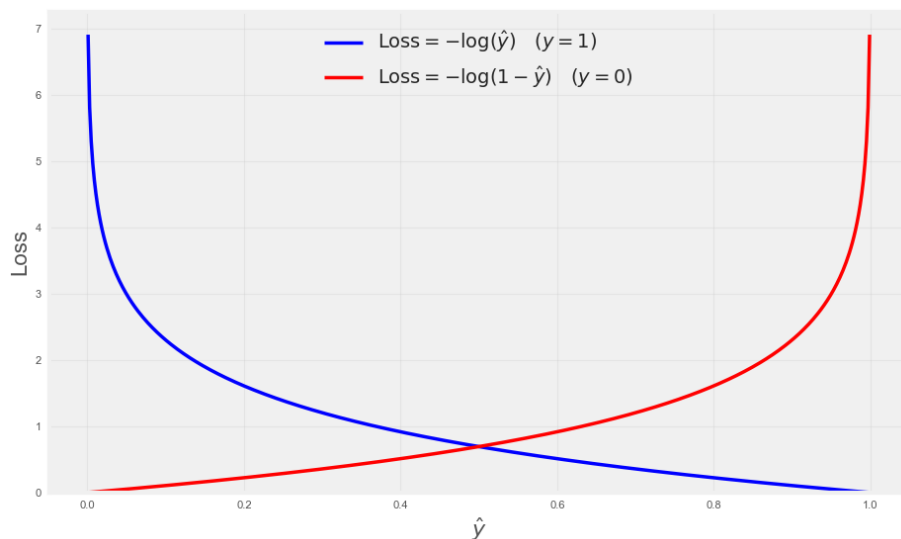
$$\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i) \in (0, 1)$$

$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \begin{cases} -\log(\hat{y}_i), & \text{if } y_i = 1, \\ -\log(1 - \hat{y}_i), & \text{if } y_i = 0 \end{cases}$$

The negative log likelihood loss function with $\sigma(z)$ makes the cost function convex.



Negative log likelihood also incentivizes appropriately: big mismatch $\implies$ big loss, small mismatch $\implies$ small loss.



$$\mathbf{z}^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)}$$

$$\text{Loss per example, } L(\mathbf{x}^{(i)}, \mathbf{w}) = -y^{(i)} \log(\sigma(\mathbf{z}^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{z}^{(i)}))$$

$$\frac{\partial \sigma(\mathbf{z}^{(i)})}{\partial \mathbf{w}} = \frac{\partial \sigma(\mathbf{z}^{(i)})}{\partial \mathbf{z}^{(i)}} \cdot \frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{w}} = \sigma(\mathbf{z}^{(i)}) (1 - \sigma(\mathbf{z}^{(i)})) \mathbf{x}^{(i)}$$

$$\frac{\partial L(\mathbf{x}^{(i)}, \mathbf{w})}{\partial \mathbf{w}} = (\sigma(\mathbf{z}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)}$$

$$\frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L(\mathbf{x}^{(i)}, \mathbf{w})}{\partial \mathbf{w}}$$

More concisely,

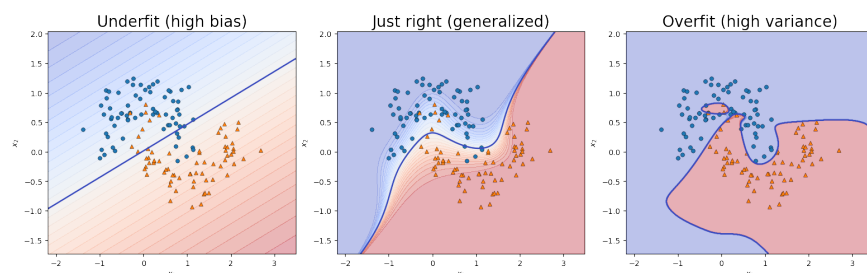
$$\mathbf{z} = \mathbf{X}\mathbf{w}$$

$$J(\mathbf{X}, \mathbf{w}) = -\frac{1}{m} [\mathbf{y}^T \log(\sigma(\mathbf{z})) + (\mathbf{1} - \mathbf{y})^T \log(1 - \sigma(\mathbf{z}))]$$

$$\frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}} = \frac{1}{m} \mathbf{X}^T (\sigma(\mathbf{z}) - \mathbf{y})$$

$$\text{Gradient descent: } \mathbf{w} = \mathbf{w} - \alpha \frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}}$$

2 Generalization



2.1 Address overfit

1. Collect more training examples keeping the model as it is.
2. Select a smaller set of informative features to simplify the model. Regularization disables features in a gradual manner.

2.1.1 Regularization

L2:

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad J(\mathbf{w}) &= \frac{1}{m} \|\text{error}(\mathbf{w})\|^2 \\ \text{gradient descent:} \quad \mathbf{w} &= \mathbf{w} - \alpha \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} \\ \underset{\mathbf{w}}{\text{minimize}} \quad J_{\text{reg}}(\mathbf{w}) &= \frac{1}{m} \left(\|\text{error}(\mathbf{w})\|^2 + \lambda \|\mathbf{w}\|^2 \right) \\ \text{gradient descent (reg):} \quad \mathbf{w} &= \underbrace{\left(1 - \alpha \cdot \frac{2\lambda}{m} \right)}_{\text{shrinkage}} \mathbf{w} - \alpha \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} \quad // \quad (\alpha \cdot 2\lambda/m) \sim 0 \end{aligned}$$

Here, $\lambda \|\mathbf{w}\|^2$ encourages smaller parameter values. With bigger λ 's, the parameters approach zero.

We can think regularization in terms of a constrained optimization problem:

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & J(\mathbf{w}) \\ \text{with constraint:} \quad & \|\mathbf{w}\|^2 \leq r^2 \end{aligned}$$

In other words, minimize $J(\mathbf{w})$ but do not let $\mathbf{w}$ go beyond a ball of radius r .

L1:

$$\underset{\mathbf{w}}{\text{minimize}} \quad J_{\text{reg}}(\mathbf{w}) = \frac{1}{m} \left(\|\text{error}(\mathbf{w})\|^2 + \lambda \|\mathbf{w}\|_1 \right)$$

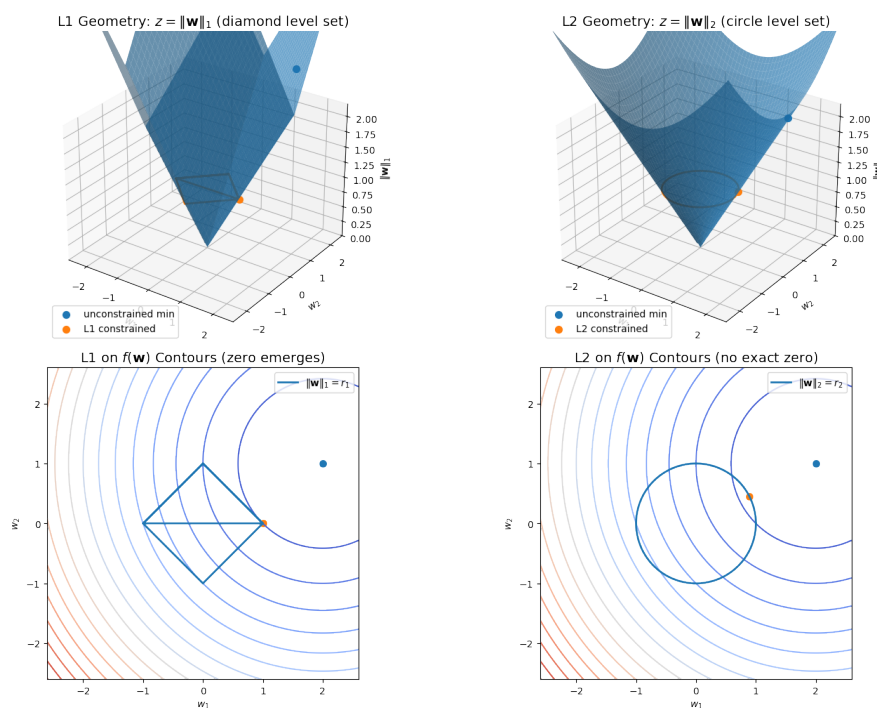
$$\|\mathbf{w}\|_1 = \sum_i |w_i|$$

$$\text{gradient descent (reg):} \quad \mathbf{w} = \mathbf{w} - \underbrace{\alpha \cdot \frac{\lambda}{m} \text{sign}(\mathbf{w})}_{\text{constant pull towards 0}} - \alpha \cdot \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$$

The corresponding constrained optimization:

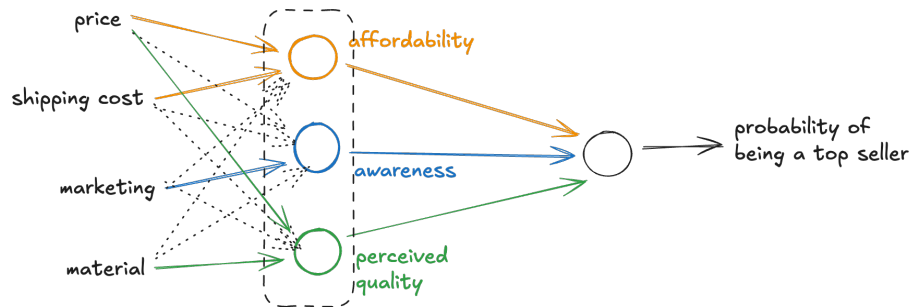
$$\underset{\mathbf{w}}{\text{minimize}} \quad J(\mathbf{w})$$

$$\text{with constraint:} \quad \|\mathbf{w}\|_1 \leq r$$



With L1 regularization, parameters for less important features can be exactly zero.

3 Neural Networks



- A neural network realizes a piecewise differentiable function.
- In multi-layer perceptron (MLP), an individual neuron can do linear or logistic regression.
- Intermediate layers of neurons pick out features in a hierarchical manner. Left to right, the features become more abstract – feature from inputs, feature from features.
- Unlike other ML algorithms, a neural network's performance scales well with data.

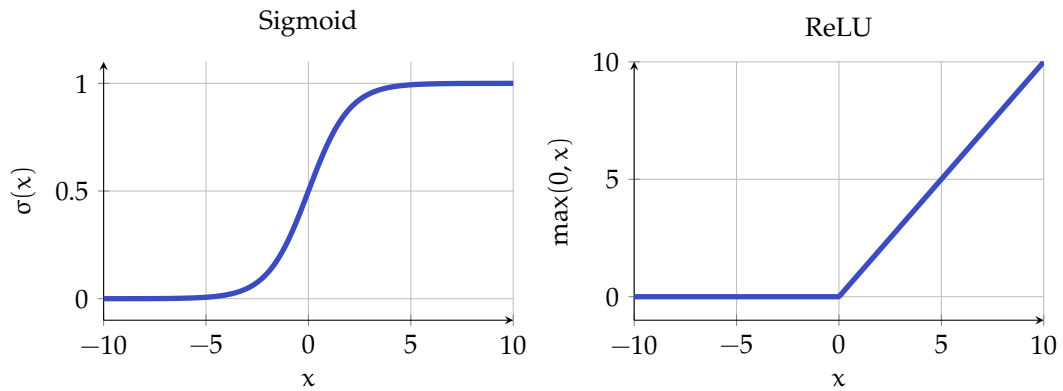
```
1 layer_1 = keras.layers.Dense(  
2     units=3,  
3     activation="sigmoid"  
4 )  
5 layer_2 = keras.layers.Dense(  
6     units=1,  
7     activation="sigmoid"  
8 )  
9 model = keras.Sequential([layer_1, layer_2])  
10 model.compile(optimizer="adam", loss="mse")  
11 model.fit(X, y, epochs=100)  
12 model.predict(X_new)
```

3.1 Loss function

Target of optimization. For binary classification, we can use "binary cross entropy". For regression, we can use "mean squared error".

3.2 Activation function

Without activation function, a neural network can only implement a linear function. For example, it cannot implement the XOR function.



- Sigmoid(x) $\in (0, 1)$ vs. ReLU(x) $\in [0, \infty)$. So, ReLU(x) allows large positive outputs.
- Output layer:
 - Binary classification: Sigmoid
 - Non-negative Regression: ReLU
 - Regression: None.
- Hidden layer: ReLU. Because:
 - Faster to compute
 - Flat regions have near-zero gradients, so gradient descent takes tiny steps and progresses slowly there. Sigmoid has two flat regions, ReLU has one.
- The off feature (0-region) of ReLU enables a neural network to stitch together linear segments to model a complex non-linear function.

3.2.1 Softmax

For multi-class classification. For n classes, activation of one of the n output neurons:

$$z_j = \mathbf{w}_j \cdot \mathbf{x} + b_j, j = 1, 2, \dots, n \quad // \text{ logits}$$

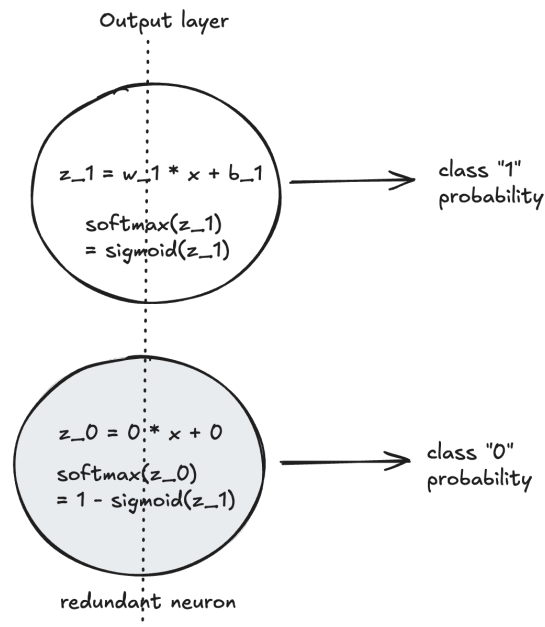
$$a_j = \Pr(y = j | \mathbf{x}) = \text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^n e^{z_k}}$$

- Softmax($z + c$) = Softmax(z)
- When $n = 2$, Softmax $\equiv$ Sigmoid:

$$a_1 = \text{Softmax}(z_1) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} = \frac{1}{1 + e^{-(z_1 - z_0)}}$$

When $z = 0$, $a_1 = \text{Sigmoid}(z_1)$, $a_0 = 1 - a_1$

As if we have two output neurons, one with all parameters set to 0.



Training logits and applying Softmax at inference time improves numerical stability:

```

1 model = Sequential(
2     [
3         Dense(units=25, activation="relu"),
4         Dense(units=15, activation="relu"),
5         # outputs logits
6         Dense(units=10, activation="linear")
7     ]
8 )
9 model.compile(
10     optimizer="adam",
11     # train logits (z's)
12     loss=SparseCategoricalCrossentropy(from_logits=True)
13 )
14 model.fit(X, y, epochs=100)
15 logits = model(X_new)
16 probabilities = tf.nn.softmax(logits)

```

3.3 Optimizer

3.3.1 Gradient descent

$$\mathbf{w} = \mathbf{w} - \alpha \nabla J(\mathbf{w})$$

Learning rate α is fixed.

3.3.2 Adam (ADaptive Moment estimation)

$$\mathbf{w} = \mathbf{w} - \mathbf{A} \nabla J(\mathbf{w}), \text{ with } \mathbf{A} = \text{diag}(\alpha_1, \alpha_2, \dots, \alpha_n)$$

- Generally faster convergence than gradient descent.
- Each parameter has its own learning rate.
- Each learning rate adapts on the fly based on earlier steps.

3.4 Layers

3.4.1 Dense layer

Activation of a neuron is a function of the activations of *all neurons* in the previous layer.

3.4.2 Convolutional layer

Activation of a neuron is a function of the activations of a *limited number of neurons* (overlapping sliding window) in the previous layer.

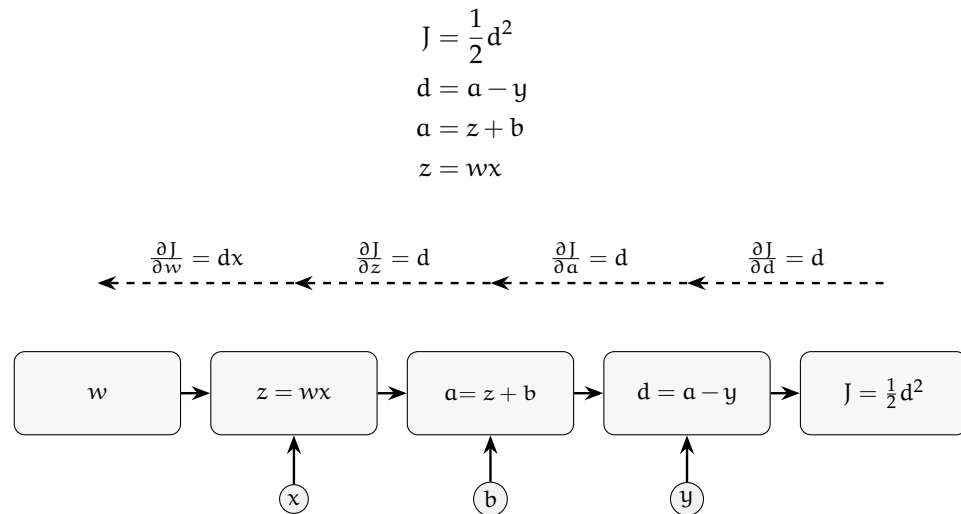
- Faster computation
- Less prone to overfitting

3.5 Backpropagation

```
1 J, w = sympy.symbols("J, w")
2 J = 1 / (1 + sympy.exp(-w))
3 # symbolic differentiation
4 dJ_dw = sympy.diff(J, w)
5 dJ_dw.subs([(w, 2)])
```

3.5.1 Computation graph

Evaluate DAG in topological order in forward-prop and in topological order of the reversed DAG in backprop.



With N nodes and P parameters, $T(\text{backprop}) = \mathcal{O}(N + P)$

3.5.2 Autodiff

Tensorflow (or PyTorch) takes care of backprop for us via automatic differentiation.

4 References

- Machine learning specialization – Coursera